

Tema 2 - CSS y construcción de interfaces web

2.4 - Presentación visual, tipografía, imágenes, efectos e iconos

De la caja a la presentación visual

Una interfaz puede tener una estructura correcta, ocupar las dimensiones previstas y encontrarse en la posición adecuada sin disponer todavía de una identidad visual definida.

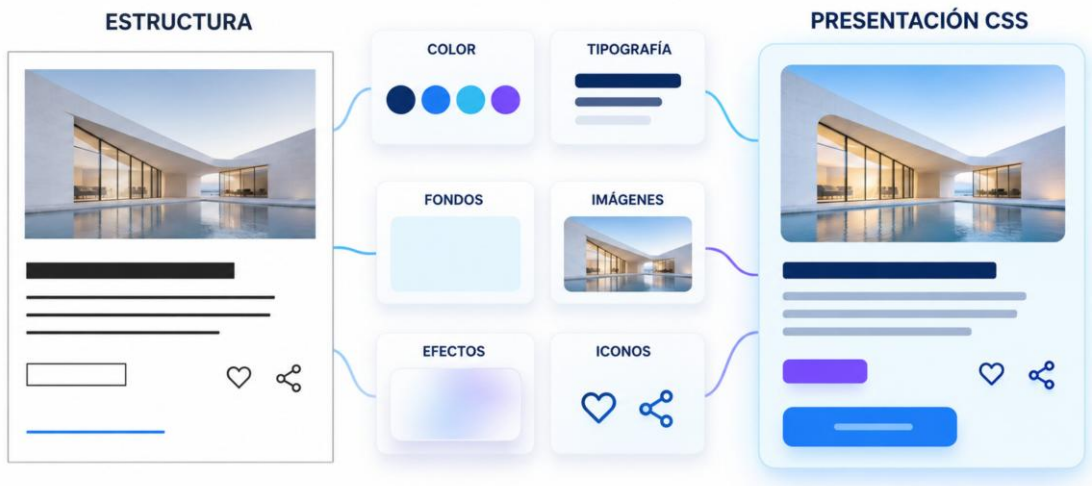
Hasta ahora se ha trabajado principalmente con la forma en que los elementos ocupan espacio: dimensiones, márgenes, rellenos, flujo, visibilidad y posicionamiento. El siguiente paso consiste en controlar cómo se representan esos elementos en pantalla. CSS permite intervenir sobre el color, los fondos, la tipografía, las imágenes, las sombras, los filtros y numerosos detalles visuales. Estas propiedades no sustituyen una estructura HTML correcta: actúan sobre ella para convertirla en una interfaz reconocible y coherente.

Esta distinción será especialmente importante al trabajar con imágenes. CSS puede recortar visualmente una fotografía, modificar su brillo o aplicarle un desenfoque, pero el archivo original permanece intacto. La creación, edición y optimización del recurso se realizará posteriormente con herramientas específicas.

> HTML aporta estructura y significado. CSS determina cómo se presenta visualmente esa estructura.

1	2	3
Color	Tipografía	Fondos
4	**5**	**6**
Imágenes	Efectos	Iconos

La composición muestra una transformación visual progresiva: de una estructura sin presentación hacia una interfaz con identidad visual completa.



Representar colores en CSS

El color puede aplicarse a numerosos componentes de una interfaz: texto, fondos, bordes, sombras, iconos y otros efectos visuales. CSS admite distintas formas de expresar un mismo color.

Los nombres predefinidos, como `blue` o `tomato`, resultan cómodos para pruebas sencillas, pero ofrecen un control limitado.

La notación hexadecimal representa los componentes de color mediante valores hexadecimales. Es frecuente encontrarla en guías de estilo y sistemas de diseño.

`rgb()` expresa el color mediante sus componentes rojo, verde y azul. `hsl()` utiliza tono, saturación y luminosidad. Esta representación puede resultar especialmente cómoda cuando interesa construir variaciones relacionadas de un mismo color.

```
.titulo { color: #172554; }

.aviso { background-color: rgb(245 158
11); }

.enlace { color: hsl(215 73% 47%); }
```

#2266CC

rgb(34 102 204)

hsl(215 73% 47%)

El mismo azul expresado con tres notaciones diferentes. El resultado visual es idéntico; la elección depende del contexto y del sistema de diseño.

> No existe una notación universalmente mejor. La elección depende del contexto, del sistema de diseño y de la forma en que resulte más sencillo mantener los colores.

Transparencia: no es lo mismo modificar un color que todo el elemento

CSS permite incorporar transparencia dentro de determinadas funciones de color mediante un canal alfa. En este caso únicamente se modifica la transparencia de ese color concreto.

```
.panel { background-color: rgb(34 102 204 / 0.18); }
```

La propiedad `opacity` funciona de manera diferente. Se aplica al resultado completo del elemento, incluido su contenido.

```
.panel { opacity: 0.5; }
```

Si un panel contiene texto, iconos o imágenes, reducir `opacity` hará también que esos descendientes se representen de forma semitransparente. Por este motivo, cuando solo se necesita un fondo translúcido suele ser preferible utilizar un color con alfa.

ALFA	OPACITY
Modifica la transparencia de un color concreto. El contenido del elemento conserva su opacidad completa.	Modifica la representación completa del elemento. Texto, iconos e imágenes se representan igualmente semitransparentes.



El fondo forma parte de la presentación, no necesariamente del contenido

`background-color` establece el color de fondo de una caja. `background-image` permite utilizar una imagen, un degradado u otro valor de tipo imagen como fondo.

```
.hero {
  background-color: #172554;
  background-image: url("../img/campus.webp");
}
```

Una imagen de fondo no debe confundirse con una imagen de contenido. Si una fotografía aporta información que forma parte del documento, normalmente debe incorporarse mediante HTML. Si únicamente construye la ambientación o decoración visual de una zona, puede tener sentido utilizarla como fondo CSS.

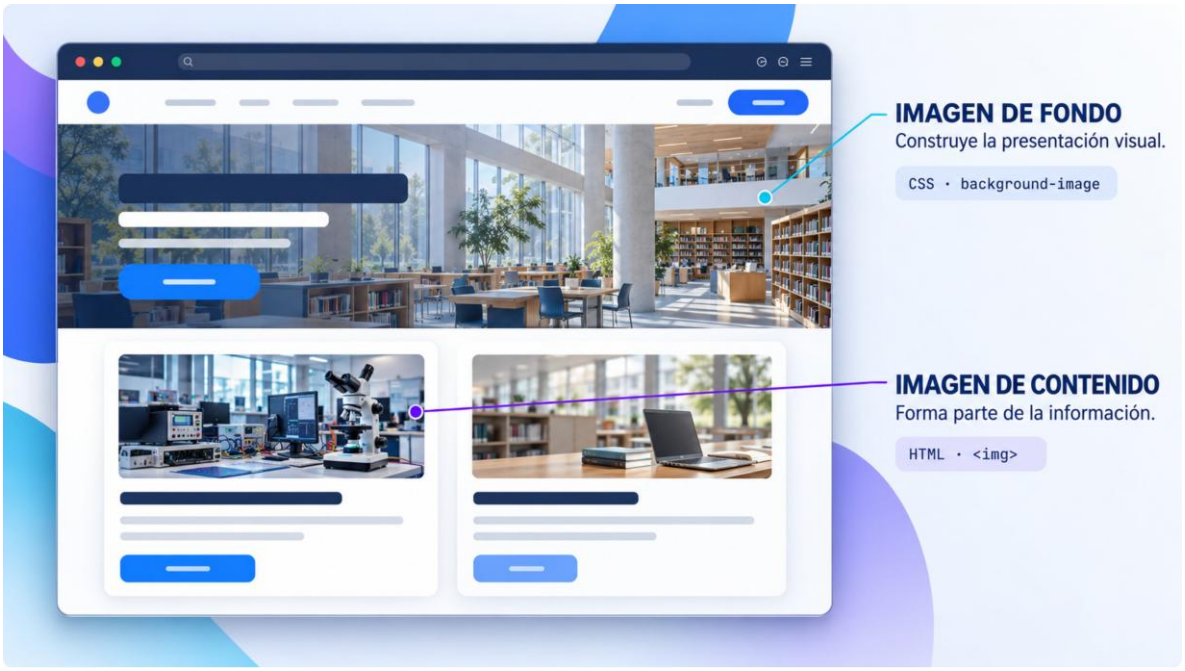
También debe tenerse en cuenta la ruta del archivo. Cuando `url()` aparece en una hoja CSS externa, una ruta relativa se interpreta respecto a la ubicación del archivo CSS, no respecto al documento HTML.

IMAGEN DE CONTENIDO

Forma parte de la información del documento. Se incorpora mediante HTML: `<img src="..." alt="...">`

IMAGEN DE FONDO

Forma parte principalmente de su presentación visual. Se incorpora mediante CSS: `background-image: url(...)`



Posición, repetición y tamaño del fondo

Una imagen de fondo no tiene por qué representarse con sus dimensiones originales ni repetirse indefinidamente.

`background-repeat` determina si la imagen se repite. `background-position` establece qué zona del fondo se utiliza como referencia. `background-size` controla el tamaño con que se representa.

```
.hero {
  background-image: url("../img/campus.webp");
  background-repeat: no-repeat;
  background-position: center;
  background-size: cover;
}
```

`cover` amplía o reduce la imagen hasta cubrir por completo la caja. Puede provocar recorte. `contain` intenta mostrar la imagen completa dentro de la caja. Puede dejar zonas libres.

La propiedad abreviada `background` permite reunir varias propiedades en una sola declaración:

```
.hero {
  background: #172554 url("../img/campus.webp") center / cover no-repeat;
}
```

Al utilizar una propiedad abreviada conviene recordar que las subpropiedades omitidas pueden recuperar sus valores iniciales. No debe utilizarse sin conocer qué valores se están sustituyendo.

ORIGINAL / REPETICIÓN

La imagen se repite horizontalmente y verticalmente con sus dimensiones originales.

CONTAIN

La imagen completa es visible dentro de la caja. Pueden quedar zonas libres a los lados o arriba y abajo.

COVER

La imagen cubre toda la caja. Parte del contenido puede quedar recortada fuera del área visible.



Crear fondos sin utilizar un archivo de imagen

Los degradados CSS generan transiciones visuales entre colores y se comportan como imágenes dentro de las propiedades de fondo.

`linear-gradient()` crea una transición siguiendo una dirección. `radial-gradient()` genera la transición desde un punto o una forma central.

Los puntos de parada permiten determinar dónde aparece cada color y cuánto espacio ocupa la transición.

```
.hero {
  background: linear-gradient(
    135deg,
    #172554,
    #2266CC 55%,
    #06B6D4
  );
}
```

También pueden combinarse varias capas de fondo. Una aplicación habitual consiste en superponer un degradado semitransparente a una fotografía para mejorar la integración visual del texto.

> Los degradados son imágenes generadas por CSS. No necesitan un archivo gráfico independiente.

```
.hero {
  background-image:
    linear-gradient(
      rgb(23 37 84 / 0.78),
      rgb(23 37 84 / 0.38)
    ),
    url("../img/campus.webp");
}
```



Un fondo puede desplazarse de forma diferente al contenido

Por defecto, el fondo de un elemento se desplaza junto con la página. `background-attachment` permite modificar este comportamiento.

```
.hero {
  background-image: url("../img/campus.webp");
  background-position: center;
  background-size: cover;
  background-attachment: fixed;
}
```

Con `fixed`, el fondo puede permanecer visualmente ligado al área de visualización mientras el contenido continúa desplazándose. Esta diferencia produce un efecto sencillo de profundidad que suele asociarse al denominado efecto **parallax**. No debe confundirse con `position: fixed`· `background-attachment` controla el comportamiento de una capa de fondo; `position` controla la posición de una caja.

El efecto debe emplearse con moderación. Su funcionamiento y rendimiento pueden variar según el navegador, el dispositivo y la forma en que esté construida la página.

background-attachment: fixed

Fija el comportamiento del fondo respecto al área de visualización. El fondo no se desplaza con el scroll.

position: fixed

Fija la posición de un elemento HTML respecto al viewport. No tiene relación con las capas de fondo.



background-blend-mode: combinar las capas de un mismo fondo

Cuando un elemento utiliza varias capas de `background`, estas pueden mezclarse en lugar de limitarse a superponerse. `background-blend-mode` determina cómo interactúan visualmente las capas de fondo del propio elemento.

```
.hero {
  background-image:
    linear-gradient(#2266CC, #7C3AED),
    url("../img/campus.webp");
  background-size: cover;
  background-blend-mode: multiply;
}
```

El modo `normal` mantiene la composición habitual. `multiply` tiende a oscurecer el resultado al combinar las capas. `screen` produce normalmente un resultado más claro. `overlay` combina características de multiplicación y aclarado en función de los tonos existentes.

No es necesario memorizar todos los modos disponibles. Lo importante es comprender que modifican matemáticamente la forma en que se combinan los colores de las distintas capas.

NORMAL

Composición habitual. Las capas se superponen sin mezcla matemática adicional.

MULTIPLY

Oscurece el resultado al multiplicar los valores de color de cada capa.

SCREEN

Produce normalmente un resultado más claro al invertir y multiplicar los valores.

OVERLAY

Combina multiplicación y aclarado en función de los tonos existentes.

> background-blend-mode mezcla las capas de background pertenecientes al mismo elemento.

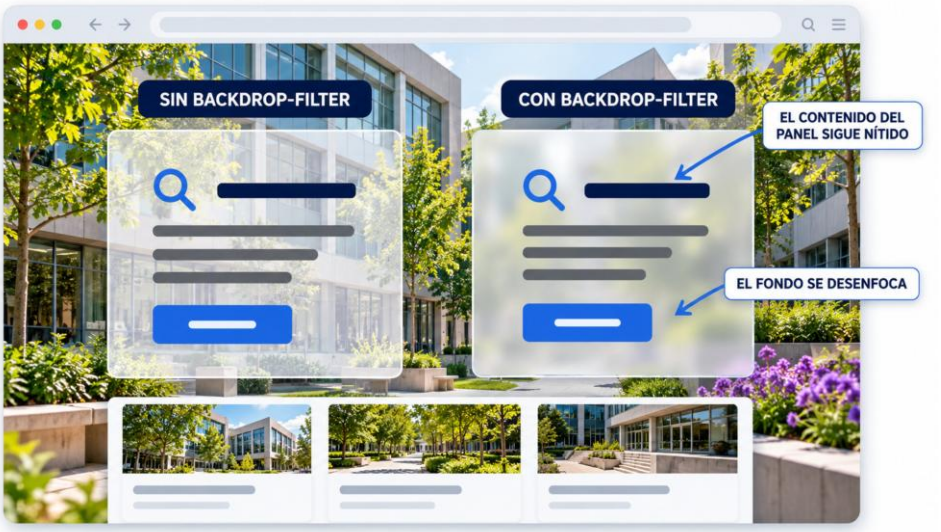


backdrop-filter: modificar visualmente el contenido situado detrás

En algunas interfaces interesa mantener visible el contenido situado detrás de un panel y, al mismo tiempo, separarlo visualmente del primer plano. `backdrop-filter` aplica funciones de filtro al contenido que queda detrás del elemento.

```
.panel {
  background-color: rgb(255 255 255 /
0.28);
  backdrop-filter: blur(12px);
}
```

El desenfoque no se aplica al contenido del propio panel. Actúa sobre lo que se observa a través de él. Para que el efecto pueda apreciarse, el panel debe permitir ver parcialmente el fondo mediante transparencia o zonas no completamente opacas.



Las funciones pueden combinarse:

```
.panel {
  background-color: rgb(255 255 255 / 0.28);
  backdrop-filter: blur(12px) saturate(120%);
}
```

filter

Modifica el propio elemento: su contenido, su fondo y su apariencia completa.

backdrop-filter

Modifica visualmente lo que existe detrás del elemento, aplicando filtros al contenido posterior.

El efecto no debe ser la única razón por la que el contenido resulte legible. El panel debe conservar una presentación suficientemente clara incluso cuando la diferencia visual sea menor.

mix-blend-mode: el propio elemento participa en la mezcla

`mix-blend-mode` controla cómo se mezcla el contenido de un elemento con aquello que queda visualmente detrás de él.

```
.titulo {
  color: white;
  mix-blend-mode: difference;
}
```

Su funcionamiento es distinto del de `background-blend-mode`. `background-blend-mode` mezcla entre sí las capas de fondo del mismo elemento. `mix-blend-mode` mezcla el elemento representado con el contenido que ya existe detrás. `backdrop-filter` no mezcla ambas capas: aplica funciones de filtro al contenido posterior antes de mostrarlo a través del elemento.

background-blend-mode

Capas de background del mismo elemento. Mezcla las capas de fondo entre sí.

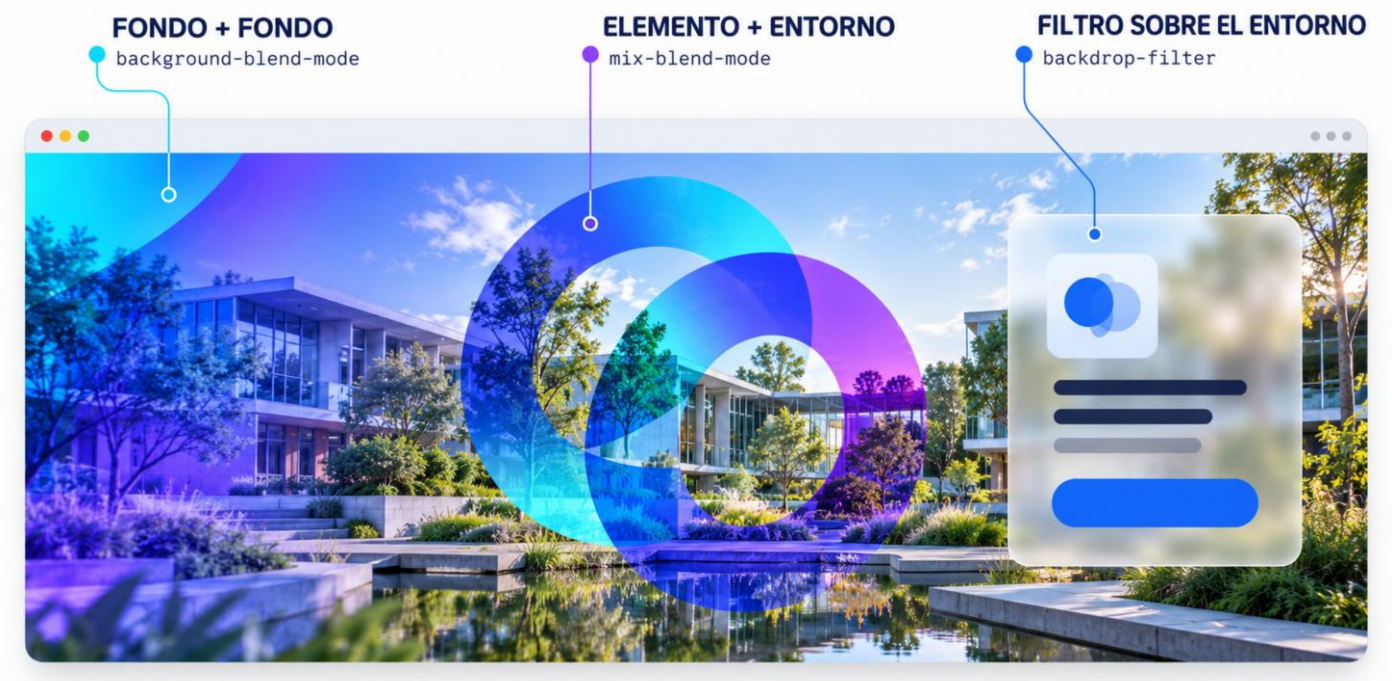
mix-blend-mode

Elemento frente al contenido situado detrás. El elemento participa en la mezcla con su entorno.

backdrop-filter

Filtrado visual del contenido situado detrás. No mezcla colores: aplica funciones gráficas al fondo posterior.

Los modos de mezcla pueden generar resultados muy dependientes de los colores y de las imágenes que haya detrás. Resultan útiles para composiciones concretas, pero no deberían emplearse si dificultan la previsibilidad o la lectura de la interfaz.



Una fuente solicitada no siempre estará disponible

`font-family` establece la familia tipográfica utilizada para representar el texto.

```
body {
  font-family: system-ui, "Segoe UI", Roboto, sans-serif;
}
```

El navegador intenta utilizar las familias en el orden indicado. Si una fuente no está disponible, continúa con la siguiente. Esta sucesión se denomina pila o lista de fuentes de reserva.

Las familias genéricas, como `serif`, `sans-serif` o `monospace`, permiten terminar la lista con una categoría que el navegador pueda resolver mediante alguna fuente disponible en el sistema. `system-ui` solicita una fuente diseñada para integrarse con la interfaz del sistema operativo.

Los nombres que contienen espacios deben escribirse entre comillas.

> font-family no garantiza que una fuente concreta exista en el dispositivo. Una pila de reserva evita depender de una única opción.

SERIF

Una misma frase representada con una fuente con serifa. Adecuada para lectura extensa.

SANS-SERIF

La misma frase sin serifa. Frecuente en interfaces digitales contemporáneas.

MONOSPACE

Cada carácter ocupa el mismo ancho. Habitual en bloques de código.

SYSTEM-UI

Fuente del sistema operativo. Integración visual con el entorno del usuario.

@font-face permite registrar una fuente propia

Una interfaz no tiene por qué depender únicamente de las fuentes instaladas en el dispositivo. `@font-face` permite declarar una fuente disponible como recurso del proyecto y asignarle un nombre que posteriormente podrá utilizarse mediante `font-family`.

Estructura del proyecto

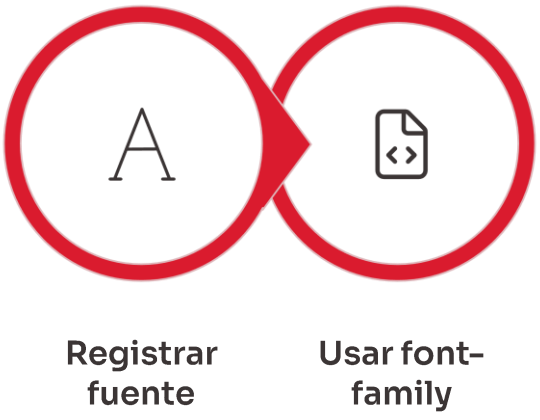
```
proyecto/
├── css/
│   └── estilos.css
├── fonts/
│   └── aula-sans-regular.woff2
└── index.html
```

La ruta relativa en `url()` se resuelve respecto a la ubicación del archivo CSS, no del documento HTML.

```css
@font-face {
  font-family: "Aula Sans";
  src: url("../fonts/aula-sans-regular.woff2")
format("woff2");
  font-weight: 400;
  font-style: normal;
  font-display: swap;
}

body {
  font-family: "Aula Sans", sans-serif;
}
```

«Aula Sans» es únicamente un nombre ficticio utilizado en el ejemplo. WOFF2 es un formato especialmente apropiado para distribuir fuentes en la web. `font-display: swap` permite que el navegador utilice inicialmente una fuente disponible y sustituya la representación cuando la fuente web termina de cargarse.



Una familia tipográfica puede contener varias fuentes reales

Un archivo de fuente no representa necesariamente todos los pesos y estilos de una familia. Si el proyecto dispone de archivos independientes para texto normal, negrita o cursiva, conviene registrarlos correctamente.

```
@font-face {
  font-family: "Aula Sans";
  src: url("../fonts/aula-sans-regular.woff2") format("woff2");
  font-weight: 400;
  font-style: normal;
}

@font-face {
  font-family: "Aula Sans";
  src: url("../fonts/aula-sans-bold.woff2") format("woff2");
  font-weight: 700;
  font-style: normal;
}

@font-face {
  font-family: "Aula Sans";
  src: url("../fonts/aula-sans-italic.woff2") format("woff2");
  font-weight: 400;
  font-style: italic;
}
```

Los tres archivos se registran con el mismo nombre de familia porque pertenecen a «Aula Sans». Los descriptores `font-weight` y `font-style` indican al navegador qué variante contiene cada archivo. Una vez aplicada la familia tipográfica, el navegador selecciona automáticamente la variante que corresponde al peso y al estilo solicitados.

```
body { font-family: "Aula Sans", sans-serif; }

h2 { font-weight: 700; }

.cita { font-style: italic; }
```

`body` aplica «Aula Sans» al documento y sus descendientes la heredan. Por tanto, el `h2` utiliza la variante registrada con peso `700` y estilo `normal`, mientras que `.cita` utiliza la variante registrada con peso `400` y estilo `italic` (cursiva).

Cada combinación de familia, peso y estilo identifica una variante concreta. Registrar correctamente los archivos permite que el navegador utilice la fuente correspondiente en lugar de simular artificialmente un peso o un estilo que no se ha proporcionado.

Aula Sans Normal · 400

Texto en peso normal. Peso de cuerpo de texto habitual en lectura continua.

Aula Sans · Negrita · 700

Texto en negrita. Utilizado para títulos, énfasis y jerarquía visual.

Aula Sans · Cursiva · 400

Variante en cursiva. Útil para citas, términos técnicos y textos secundarios.

Una fuente también puede cargarse desde un servicio externo

Otra posibilidad consiste en utilizar un servicio especializado que proporcione las hojas de estilo y los archivos de fuente. Google Fonts es un ejemplo habitual.

Cuando una fuente se obtiene desde un servicio externo, primero es necesario incorporar al documento los recursos que proporciona ese servicio. En el caso de Google Fonts, los elementos `<link>` generados deben colocarse dentro del `<head>` del documento HTML.

Incorporar la fuente en el HTML

```
<head>

  <link rel="preconnect" href="https://fonts.googleapis.com">

  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600;700&display=swap"
  rel="stylesheet">

</head>
```

El último `<link>` carga la hoja de estilos proporcionada por Google Fonts. En ella se encuentran las declaraciones necesarias para utilizar la familia y los pesos seleccionados.

Utilizar la fuente desde CSS

Una vez cargada la fuente en el documento HTML, puede utilizarse desde la hoja CSS mediante `font-family`.

```
body {
  font-family: "Inter", sans-serif;
}
```

El código exacto proporcionado por un servicio externo puede cambiar. Debe utilizarse siempre el código generado o documentado por el propio proveedor. Alojar la fuente dentro del proyecto proporciona un mayor control sobre los recursos utilizados, mientras que recurrir a un servicio externo simplifica su incorporación, pero introduce una dependencia de red y del proveedor.

FUENTE LOCAL	SERVICIO EXTERNO
El proyecto distribuye el archivo. El navegador lo solicita al mismo servidor. Mayor control sobre los recursos.	El navegador solicita la fuente a un proveedor externo. Incorporación más sencilla, pero dependencia de red.

Tamaño, peso, estilo e interlineado trabajan juntos

Una jerarquía tipográfica no depende únicamente del tamaño de letra. `font-size` establece el tamaño. `font-weight` controla el peso. `font-style` permite solicitar variantes como `italic`. `line-height` determina la altura de cada línea de texto.

```
h1 {
  font-size: 2.5rem;
  font-weight: 700;
  line-height: 1.1;
}

h2 {
  font-size: 1.6rem;
  font-weight: 600;
  line-height: 1.2;
}

p {
  font-size: 1rem;
  font-weight: 400;
  line-height: 1.6;
}
```



Una `line-height` sin unidad se calcula en relación con el tamaño de fuente del propio elemento y resulta especialmente cómoda para mantener proporciones coherentes cuando cambia `font-size`.

Una diferencia tipográfica útil debe responder a una función: identificar títulos, organizar niveles y facilitar la lectura. Utilizar tamaños, pesos o estilos diferentes sin una jerarquía coherente genera ruido visual.

TÍTULO PRINCIPAL

2.5rem · 700 · line-height 1.1. Alta presencia visual, sin interlineado excesivo.

SUBTÍTULO

1.6rem · 600 · line-height 1.2. Diferenciación clara del nivel jerárquico.

PÁRRAFO

1rem · 400 · line-height 1.6. Interlineado generoso para facilitar la lectura continua.

METADATO

0.875rem · 400. Información secundaria con menor peso visual.

CSS permite intervenir en numerosos detalles del texto

Además de la familia y el tamaño, existen propiedades que afectan a la forma en que se presenta el contenido textual.

```
.introduccion {
  text-align: center;
  letter-spacing: 0.01em;
}

.enlace {
  text-decoration: underline;
}

.etiqueta {
  text-transform: uppercase;
  letter-spacing: 0.08em;
}

.parrafo {
  text-indent: 1.5em;
}
```

`text-align` determina la alineación del contenido en línea dentro de su caja. `text-decoration` controla decoraciones como el subrayado. `text-transform` modifica la presentación de mayúsculas y minúsculas sin cambiar el texto escrito en HTML. `letter-spacing` ajusta el espacio entre caracteres. `word-spacing` modifica la separación entre palabras. `text-indent` permite desplazar la primera línea de un bloque de texto.

> Estas propiedades modifican la presentación. No deben utilizarse para sustituir una estructura correcta ni para alterar el significado del contenido.

01

text-align: center

Alineación centrada del bloque. Útil para titulares y elementos aislados.

02

text-transform: uppercase

PRESENTACIÓN EN MAYÚSCULAS sin modificar el HTML original.

03

letter-spacing: 0.08em

Espaciado entre caracteres. Combina bien con texto en mayúsculas.

04

text-indent: 1.5em

Primera línea desplazada. Convención editorial para texto continuo.

CSS puede distribuir texto continuo en varias columnas

El módulo de columnas múltiples permite que un bloque de contenido textual fluya automáticamente de una columna a la siguiente.

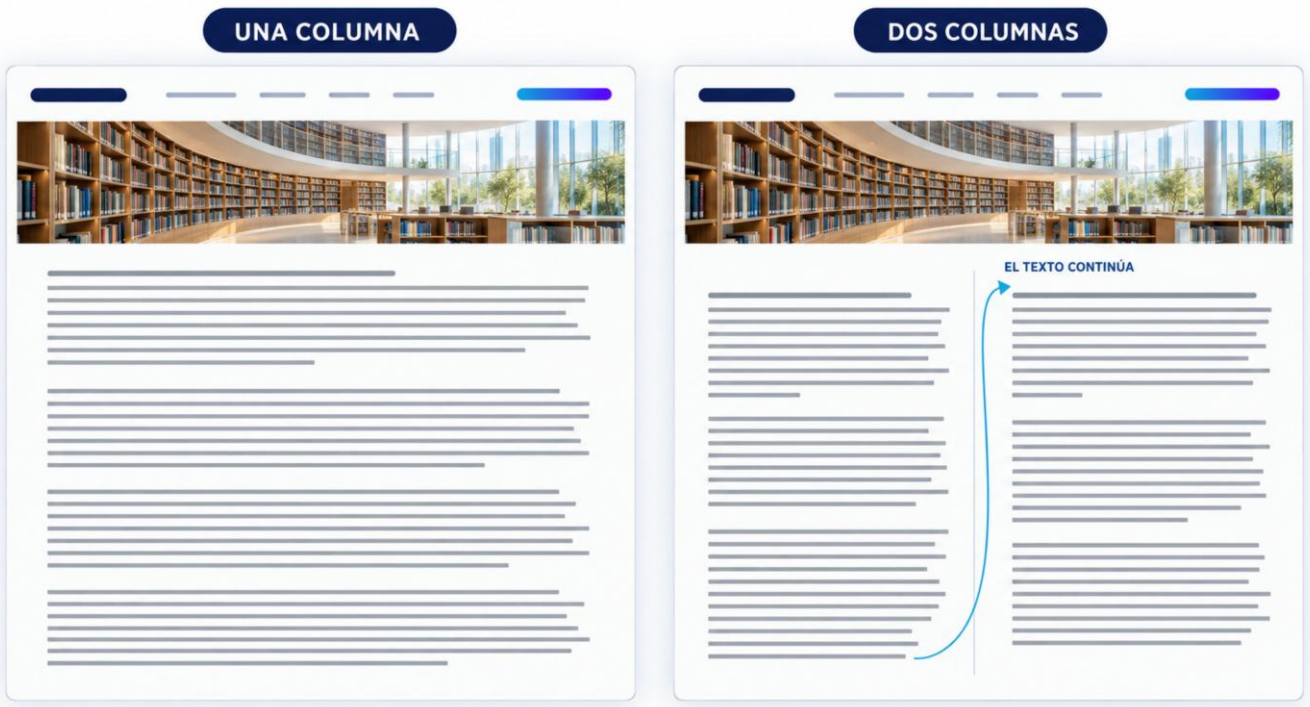
```
.articulo {
  column-count: 2;
  column-gap: 2rem;
  column-rule: 1px solid #E2E8F0;
}

.articulo {
  column-width: 18rem;
  column-gap: 2rem;
}
```

`column-count` establece una cantidad deseada de columnas. `column-width` permite proponer una anchura aproximada. `column-gap` controla la separación. `column-rule` dibuja una línea entre columnas.

FLUJO DE TEXTO: La segunda columna continúa donde termina la primera. El contenido fluye de forma automática.

No se trata de un sistema general de maquetación de interfaces. Su finalidad principal es distribuir un flujo de contenido, especialmente texto, en varias columnas. Flexbox y Grid resolverán posteriormente otros problemas de distribución mucho más amplios.



Un enlace cambia de estado durante la interacción

Los enlaces pueden representarse de manera diferente según su estado. `:link` selecciona enlaces todavía no visitados. `:visited` permite distinguir enlaces visitados, aunque los navegadores limitan determinadas propiedades por razones de privacidad. `:hover` representa el estado producido al situar un puntero sobre el enlace. `:focus-visible` permite mostrar una indicación de foco cuando resulta necesaria para la interacción mediante teclado u otros mecanismos. `:active` representa el momento en que el enlace se está activando.

```
a:link          { color: #2266CC; }
a:visited       { color: #7C3AED; }
a:hover         { text-decoration-thickness: 2px; }
a:focus-visible {
  outline: 3px solid #F59E0B;
  outline-offset: 3px;
}
a:active        { color: #172554; }
```

Eliminar la indicación visual de foco sin proporcionar una alternativa reconocible dificulta la utilización del enlace mediante teclado.



Las listas pueden cambiar de apariencia sin perder su estructura

Las listas HTML mantienen su función semántica aunque CSS modifique sus marcadores. `list-style-type` determina el tipo de marcador. `list-style-position` controla si el marcador forma parte del área interna del texto o queda fuera de ella. `list-style` reúne varias propiedades relacionadas.

```
.lista { list-style: square outside; }
```

También puede personalizarse directamente el marcador mediante `::marker`.

```
.lista li::marker {
  color: #2266CC;
  font-weight: 700;
}
```

En determinadas interfaces puede eliminarse el marcador visual:

```
.menu {
  list-style: none;
  padding: 0;
}
```

Eliminar el marcador mediante CSS no convierte la lista en otra estructura. Los elementos `ul`, `ol` y `li` siguen representando una relación de lista en el documento.

MARCADOR ESTÁNDAR

La lista utiliza el marcador predeterminado del navegador. La estructura semántica es correcta.

MARCADOR PERSONALIZADO

El pseudoelemento `::marker` aplica color y peso al marcador sin alterar la estructura HTML.

NAVEGACIÓN SIN MARCADOR

`list-style: none` elimina el marcador visual. La lista sigue siendo una lista semánticamente.



Una tabla de datos también necesita jerarquía visual

Cuando la información se relaciona mediante filas y columnas, una tabla puede mejorar su lectura mediante CSS sin perder su estructura.

```
table {
  width: 100%;
  border-collapse: collapse;
  table-layout: fixed;
}

caption {
  caption-side: top;
}

th, td {
  padding: 0.8rem;
  border-bottom: 1px solid #E2E8F0;
  text-align: left;
}

tbody tr:nth-child(even) {
  background-color: #F8FAFC;
}
```

`border-collapse` controla cómo se representan los bordes contiguos. `caption-side` determina la posición del `caption`. `table-layout` modifica el algoritmo utilizado para distribuir la anchura de las columnas.

Las diferencias visuales entre encabezados y celdas de datos deben reforzar la estructura existente en HTML, no sustituirla.

> Una tabla debe utilizarse para datos tabulares, no como mecanismo general de maquetación de una página.



Una imagen debe integrarse en la caja que la contiene

Una imagen incorporada mediante HTML conserva inicialmente sus dimensiones intrínsecas. Una regla sencilla permite evitar que una imagen sea más ancha que el espacio disponible:

```
img {
  max-width: 100%;
  height: auto;
}
```

`max-width: 100%` limita la anchura máxima al tamaño disponible. `height: auto` conserva la proporción cuando cambia la anchura. En algunos componentes también resulta conveniente:

```
img {
  display: block;
}
```

Las imágenes son elementos en línea (`inline`) de forma predeterminada. `display: block` puede eliminar el pequeño espacio asociado a la línea de texto cuando la imagen se utiliza como una pieza visual independiente.

Estas reglas no constituyen por sí solas una estrategia completa de imágenes adaptables (responsive). La selección de archivos diferentes según dispositivo se estudiará posteriormente al integrar recursos multimedia.

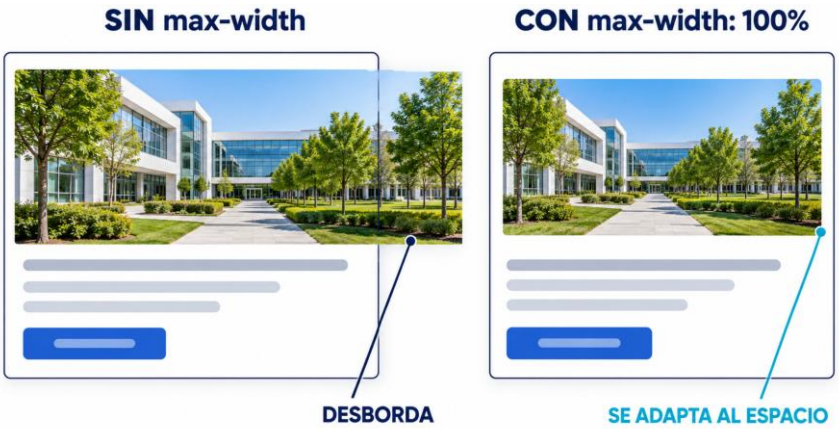
> CSS controla cómo se representa la imagen. El archivo sigue siendo el mismo.

Sin max-width

La imagen conserva sus dimensiones intrínsecas y puede desbordar el contenedor cuando este es más estrecho.

Con max-width: 100%

La imagen se limita a la anchura disponible. `height: auto` mantiene la proporción original.



Cuando la caja y la imagen tienen proporciones diferentes

En algunos componentes se necesita que todas las imágenes ocupen una caja de dimensiones comunes, aunque sus proporciones originales sean diferentes. `object-fit` determina cómo se adapta el contenido reemplazado, como una imagen, dentro de esa caja.

```
.imagen-curso {
  width: 100%;
  height: 220px;
  object-fit: cover;
}
```

fill

Rellena la caja y puede deformar la imagen para adaptarse exactamente a las dimensiones.

contain

Muestra la imagen completa sin deformarla, aunque puedan quedar zonas libres dentro de la caja.

cover

Cubre toda la caja sin deformar la imagen, aunque parte de ella pueda quedar recortada.

none

Mantiene el tamaño propio del contenido sin ningún ajuste.

scale-down

Utiliza el resultado más pequeño entre none y contain.



El recorte también puede dirigirse

Cuando `object-fit: cover` recorta parte de una imagen, el navegador necesita determinar qué zona debe conservarse como referencia. `object-position` permite modificar ese punto.

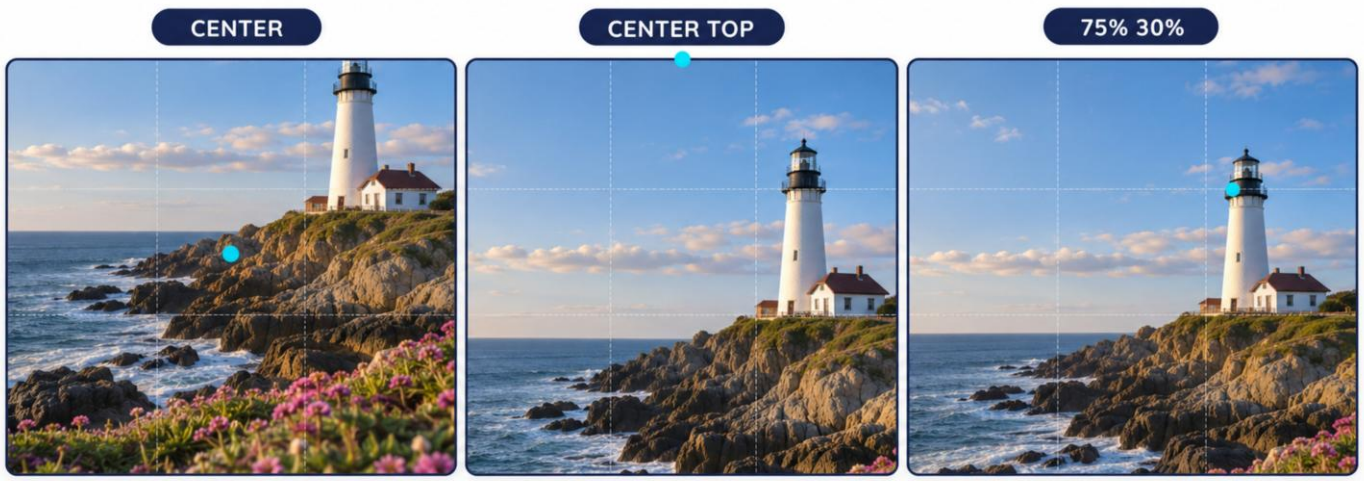
```
.imagen-curso {
  width: 100%;
  height: 220px;
  object-fit: cover;
  object-position: center top;
}

object-position: center;
object-position: 75% 30%;
object-position: left center;
```

`object-fit` y `object-position` actúan sobre una imagen u otro contenido reemplazado situado dentro de su propia caja. `background-size` y `background-position` realizan una función conceptualmente parecida sobre una imagen utilizada como fondo CSS.

IMAGEN COMO CONTENIDO	IMAGEN COMO FONDO CSS
`object-fit` + `object-position` Controlan el ajuste y el punto de referencia del elemento de imagen en HTML.	`background-size` + `background-position` Controlan el ajuste y la posición de una capa de fondo declarada en CSS.

> El mecanismo depende de dónde se encuentre la imagen: como contenido HTML o como capa de fondo CSS.



Modificar la representación de una imagen sin editar el archivo

La propiedad `filter` aplica funciones gráficas al resultado visual de un elemento. El archivo original no se modifica.

```
.imagen { filter: grayscale(1); }
```

grayscale()

Reduce o elimina el color.

sepia()

Aplica una tonalidad semejante a la fotografía sepia.

saturate()

Modifica la intensidad del color.

brightness()

Ajusta el brillo.

contrast()

Modifica el contraste.

blur()

Aplica desenfoque.

hue-rotate()

Desplaza los tonos de color.

invert()

Invierte los colores.

opacity()

Modifica la opacidad mediante la cadena de filtros.

```
.gris    { filter: grayscale(1); }
.suave   { filter: blur(4px); }
.intensa { filter: saturate(1.4); }
```

filter modifica la representación que produce el navegador. Para cambiar de forma permanente el archivo será necesario editar el recurso original.



Las funciones de filter pueden formar una cadena

Una misma declaración `filter` puede contener varias funciones separadas por espacios. Cada función recibe el resultado producido por la función anterior.

```
.portada {
  filter: contrast(1.08) saturate(1.15) brightness(0.96);
}
```

La combinación permite construir ajustes más específicos sin modificar el archivo. El orden de las funciones forma parte del resultado, ya que el procesamiento se realiza secuencialmente.



No conviene convertir los filtros en una corrección automática para cualquier fotografía. Un tratamiento excesivo puede degradar la imagen, reducir la coherencia visual o dificultar la lectura del contenido superpuesto.

```
.sin-filtro { filter: none; }
```

> Una cadena de filtros es un proceso ordenado, no una colección independiente de efectos.

La sombra puede seguir la caja o la forma visible

`box-shadow` dibuja una sombra relacionada con la caja del elemento.

```
.logo {
  box-shadow: 0 8px 18px rgb(15 23 42 / 0.25);
}
```

`drop-shadow()` es una función de `filter` y puede ajustarse a la forma visible del contenido, incluidas sus zonas transparentes.

La diferencia resulta especialmente visible en imágenes PNG o SVG con transparencia. `drop-shadow()` no proporciona todas las posibilidades de `box-shadow`. No dispone, por ejemplo, de un parámetro equivalente de expansión ni de una variante `inset`.

```
.logo {
  filter: drop-shadow(
    0 8px 10px rgb(15 23 42 / 0.28)
  );
}
```

box-shadow

Sigue la caja del elemento. La sombra siempre tiene forma rectangular o la del border-radius aplicado.

drop-shadow()

Puede seguir la silueta visible del contenido. Respeta las zonas transparentes de PNG y SVG.



La sombra también puede construir profundidad y jerarquía

`box-shadow` admite desplazamiento horizontal, desplazamiento vertical, desenfoque, expansión y color.

```
.tarjeta {
  box-shadow: 0 10px 30px rgb(15 23 42 / 0.12);
}
```

También puede utilizarse `inset` para representar una sombra hacia el interior.

```
.campo {
  box-shadow: inset 0 1px 3px rgb(15 23 42 / 0.16);
}
```

Varias sombras pueden separarse mediante comas.

```
.tarjeta {
  box-shadow:
    0 1px 2px rgb(15 23 42 / 0.08),
    0 12px 30px rgb(15 23 42 / 0.12);
}
```

`text-shadow` utiliza una sintaxis relacionada para aplicar sombra al texto.

```
.hero h1 {
  text-shadow: 0 2px 8px rgb(0 0 0 / 0.35);
}
```

Las sombras funcionan mejor cuando apoyan una jerarquía visual existente. Si todos los elementos proyectan sombras intensas, dejan de aportar información sobre profundidad o separación.

Ejemplos de sombras

Recuerda:

SOMBRA SUAVE

`box-shadow: 0 10px 30px rgb(...)`
Eleva visualmente una tarjeta sobre el fondo.

SOMBRA INTERIOR

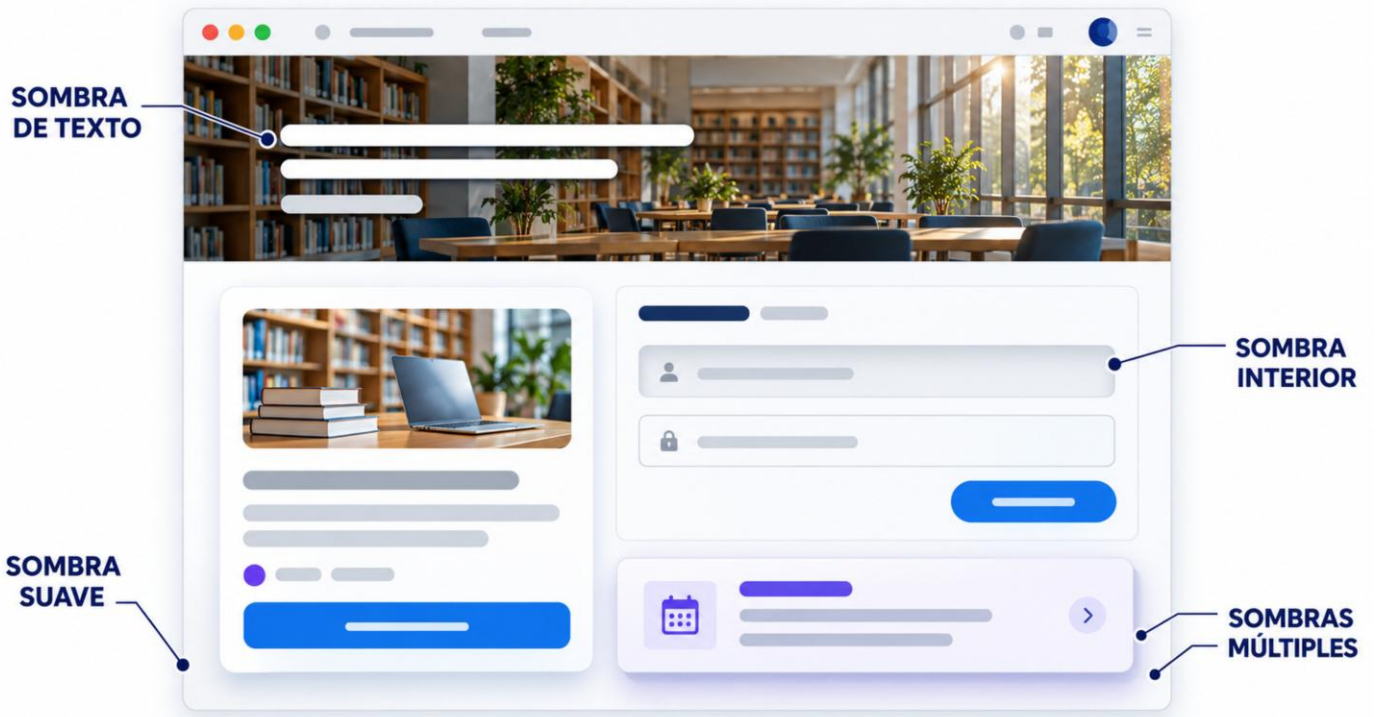
`inset`
Indica que el elemento está hundido o es editable.

SOMBRAS MÚLTIPLES

Varias capas separadas por comas. Permiten mayor control y naturalidad.

SOMBRA DE TEXTO

`text-shadow`
Aplica sombra directamente a los caracteres de texto.



El cursor puede reforzar el tipo de interacción esperado

La propiedad `cursor` permite elegir la apariencia del puntero cuando se encuentra sobre un elemento.

```
.enlace        { cursor: pointer; }
.texto-editable { cursor: text; }
.arrastrable    { cursor: grab; }
.no-disponible  { cursor: not-allowed; }
```



pointer

Indica que el elemento puede activarse, como un enlace o un botón.



text

Indica que el contenido es texto seleccionable o editable.



grab / grabbing

Indica que el elemento puede arrastrarse o está siendo arrastrado.



not-allowed

Indica que la acción no está disponible en ese momento.

Cambiar el cursor no transforma un elemento en un control interactivo. Un `div` con `cursor: pointer` continúa siendo un `div`. Del mismo modo, `cursor: not-allowed` no deshabilita realmente un control. La estructura HTML y el comportamiento deben ser correctos independientemente de la representación del cursor.

> cursor comunica una expectativa visual. No crea semántica ni comportamiento.

Font Awesome como ejemplo de biblioteca de iconos

Los iconos utilizados por una interfaz pueden proceder de una biblioteca especializada. Font Awesome proporciona conjuntos de iconos que pueden incorporarse a un proyecto mediante distintos métodos.

Instalación local

```
fontawesome/
├── css/
│   ├── fontawesome.css
│   ├── solid.css
│   └── brands.css
└── webfonts/
    └── ...
```

La estructura exacta y los nombres disponibles dependen de la distribución y versión utilizada. Debe consultarse la documentación correspondiente al paquete instalado.

```
<link rel="stylesheet"

href="fontawesome/css/fontawesome.css">
<link rel="stylesheet"
  href="fontawesome/css/solid.css">
```

Mediante un Kit externo

```
<script
src="https://kit.fontawesome.com/
YOUR_KIT_CODE.js"
crossorigin="anonymous"></script>
```

`YOUR_KIT_CODE` representa únicamente un marcador. El código real debe obtenerse desde la cuenta o documentación del proveedor.

Otra posibilidad consiste en utilizar un Kit proporcionado por el servicio. El código exacto depende de la cuenta y de la configuración realizada en el panel del proveedor.



Iconos de interfaz

Casa, menú, búsqueda, usuario.



Iconos de navegación

Flechas,indicadores direccionales (chevrons), enlaces externos.



Iconos de marca

Redes sociales, plataformas y servicios.

Un icono también participa en el sistema visual

Una vez incorporada la biblioteca, los iconos pueden insertarse mediante las clases proporcionadas por ella. Al representarse como parte de la interfaz, pueden ajustarse mediante propiedades CSS relacionadas con texto y color.

```
.icono {
  font-size: 1.25rem;
  color: #2266CC;
}
```

Una técnica especialmente útil consiste en permitir que el icono herede el color del componente.

```
.boton {
  color: #2266CC;
}

.boton .icono {
  color: currentColor;
  margin-inline-end: 0.5rem;
}
```

`currentColor` representa el valor calculado de la propiedad `color` del elemento. Facilita que texto e icono mantengan una apariencia coherente sin repetir el mismo color en varias reglas.

> Los iconos deben formar parte del sistema visual, no convertirse en adornos independientes.

Enlace con icono

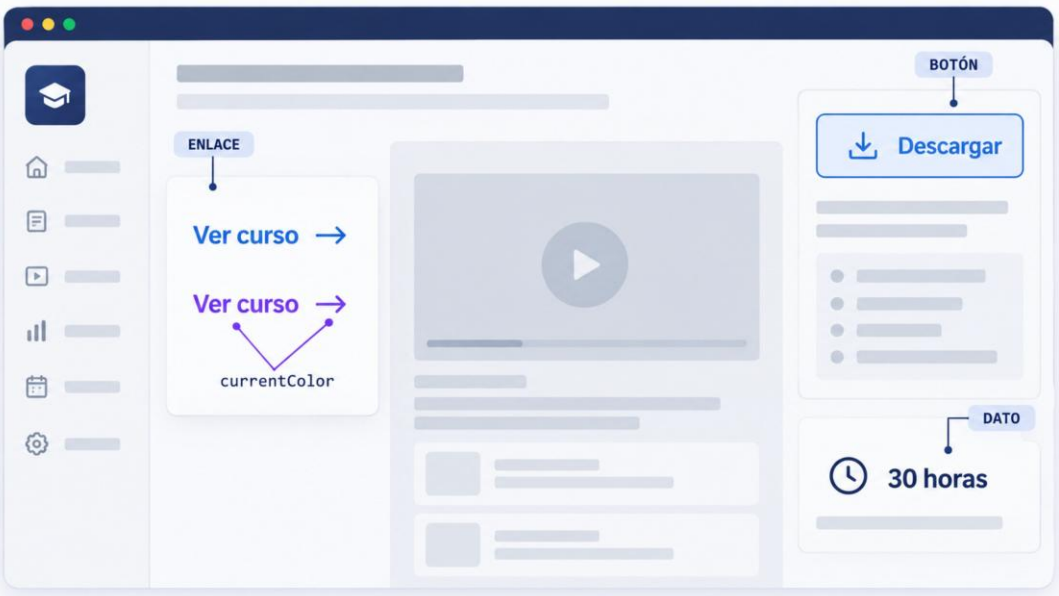
El icono acompaña el texto del enlace. `currentColor` mantiene la coherencia cuando cambia el color del enlace.

Botón con icono

El icono precede al texto de la acción. Ambos heredan el color del componente padre.

Dato con icono

El icono identifica visualmente el tipo de información. Refuerza sin sustituir el texto.



Distinguir entre iconos decorativos e iconos que representan una acción

Un icono situado junto a un texto puede ser únicamente decorativo. En ese caso, el significado ya aparece escrito y no es necesario que el icono lo repita.

```
<a href="/cursos">
  <i class="fa-solid fa-book" aria-hidden="true"></i>
  Ver cursos
</a>
```

Cuando un control está formado únicamente por un icono, el significado no puede depender exclusivamente de su aspecto visual.

```
<button aria-label="Descargar">
  <i class="fa-solid fa-download" aria-hidden="true"></i>
</button>
```

Siempre que resulte razonable, un texto visible ofrece una identificación más directa:

```
<button>
  <i class="fa-solid fa-download" aria-hidden="true"></i>
  Descargar
</button>
```

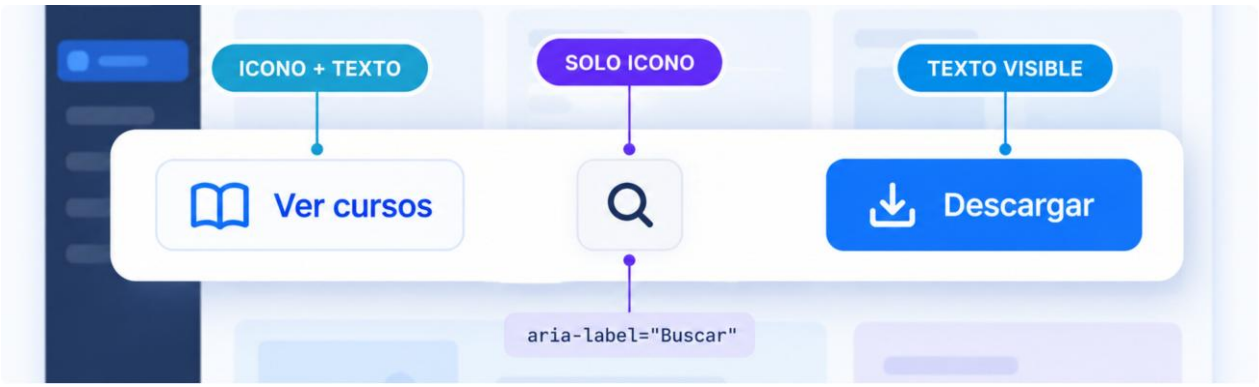
Icono + texto

El texto ya comunica el significado. El icono es decorativo y puede marcarse con `aria-hidden="true"`.

Solo icono

El control necesita una identificación comprensible. Se utiliza `aria-label` en el elemento interactivo.

Estas reglas constituyen únicamente una introducción práctica. La accesibilidad completa de los componentes se desarrollará posteriormente.



Caso integrado · La estructura antes de la apariencia

Una tarjeta de curso permite reunir propiedades estudiadas en este apartado con conceptos anteriores de cajas y posicionamiento. Antes de escribir el CSS, la estructura debe expresar correctamente el contenido.

```
<article class="curso-card">
  <div class="curso-card__media">
    <img src="img/diseno-web.webp"
    alt="Interfaz de un proyecto web">
    <span class="curso-card__badge">
      Destacado
    </span>
  </div>
  <div class="curso-card__contenido">
    <h2>Diseño de interfaces web</h2>
    <p>Diseña y construye interfaces claras,
       consistentes y actuales.</p>
    <ul class="curso-card__datos">
      <li>
        <i class="fa-solid fa-clock"
           aria-hidden="true"></i>
        30 horas
      </li>
      <li>
        <i class="fa-solid fa-laptop"
          aria-hidden="true">
        </i>
        Modalidad en línea
      </li>
    </ul>
    <a href="curso-diseno.html"
    class="curso-card__enlace">Ver curso
      <i class="fa-solid fa-arrow-right"
      aria-hidden="true">
      </i>
    </a>
  </div>
</article>
```

El HTML identifica una unidad de contenido, su imagen, estado, título, descripción, datos y acción. Ninguna de estas relaciones depende todavía de sombras, colores o efectos.

article

Unidad de contenido independiente

img

Imagen de contenido con alt

h2

Título del curso

ul / li

Datos estructurados como lista

a

Acción: enlace al curso

Caso integrado · La presentación reúne decisiones de varios apartados

```css
.curso-card {
  max-width: 25rem;
  background: white;
  border: 1px solid #E2E8F0;
  border-radius: 1rem;
  overflow: hidden;
  box-shadow: 0 12px 30px rgb(15 23 42 / 0.10);
}
.curso-card__media { position: relative; }
.curso-card__media img {
  width: 100%;
  height: 13rem;
  object-fit: cover;
  display: block;
}
.curso-card__badge {
  position: absolute;
  inset: 1rem auto auto 1rem;
  padding: 0.35rem 0.7rem;
  color: white;
  background: rgb(23 37 84 / 0.72);
  backdrop-filter: blur(6px);
  border-radius: 999px;
}
.curso-card__contenido { padding: 1.25rem; }
```

```css
.curso-card h2 {
  margin-top: 0;
  color: #172554;
  font-size: 1.35rem;
  line-height: 1.2;
}
.curso-card__datos {
  list-style: none;
  padding: 0;
}
.curso-card__datos i {
  color: #2266CC;
  margin-inline-end: 0.4rem;
}
.curso-card__enlace {
  color: #2266CC;
  font-weight: 600;
}
.curso-card__enlace:focus-visible {
  outline: 3px solid #F59E0B;
  outline-offset: 3px;
}

.curso-card__enlace i {
  color: currentColor;
  margin-inline-start: 0.4rem;
}
```

El resultado combina decisiones estudiadas en distintos momentos: dimensiones y modelo de cajas; `overflow`; posicionamiento relativo y absoluto; color; fondo translúcido; `backdrop-filter`; tipografía; `object-fit`; sombra; iconos; estados de interacción. No es necesario aplicar un filtro a una imagen únicamente porque CSS permita hacerlo. Cada propiedad debe responder a una necesidad visual concreta.

OBJECT-FIT

La fotografía cubre la zona de imagen sin deformarse.

BACKDROP-FILTER

El distintivo aplica desenfoque al contenido situado detrás.

SOMBRA

box-shadow eleva la tarjeta sobre el fondo.

TIPOGRAFÍA

Jerarquía clara entre h2, párrafo y enlace.

ICONOS

El icono del enlace utiliza `currentColor` y conserva el mismo color que el texto.

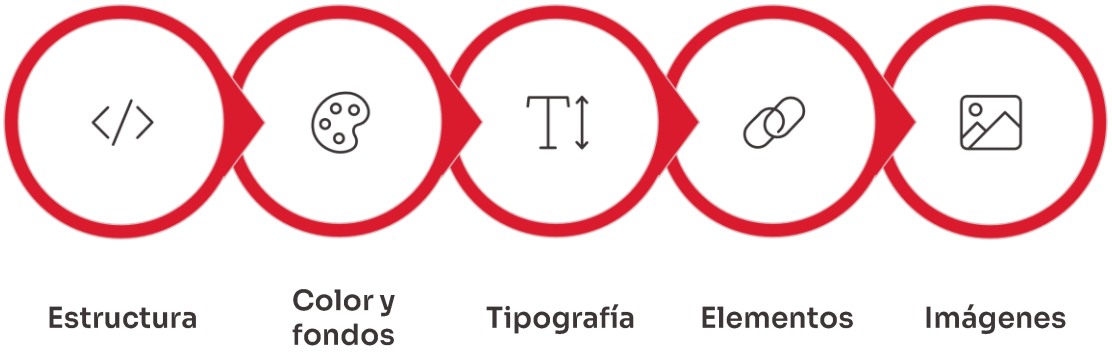
FOCO

:focus-visible con contorno visible para interacción con teclado.

Muestra del caso integrado



De una estructura funcional a una interfaz visualmente terminada



COLOR Y FONDOS

CSS permite definir colores, transparencias, fotografías, degradados, capas y modos de mezcla.

TIPOGRAFÍA

Las familias, pesos, tamaños, estilos, interlineados y ajustes de texto construyen jerarquía y facilitan la lectura.

ELEMENTOS

Enlaces, listas y tablas conservan su estructura mientras CSS modifica su representación.

IMÁGENES

Las dimensiones, object-fit y object-position permiten integrar correctamente una imagen en la caja que la contiene.

EFECTOS

filter, backdrop-filter, modos de mezcla y sombras modifican la composición visual sin alterar el archivo original.

ICONOGRAFÍA

Los iconos complementan las acciones y la información cuando se integran de forma coherente y conservan un significado comprensible.

> CSS no sustituye la estructura del documento. La presenta.

En el siguiente apartado se estudiarán componentes y controles que HTML proporciona de forma nativa. Posteriormente se abordará cómo transformar, hacer transiciones y animar los elementos de la interfaz.

Fuentes técnicas

W3C / CSS Working Group — CSS Color Module · CSS Backgrounds and Borders Module · CSS Images Module · Compositing and Blending · Filter Effects Module Level 1 · CSS Fonts Module · CSS Text Module · CSS Multi-column Layout Module · CSS Lists and Counters Module · CSS Tables Module · CSS Basic User Interface Module

MDN Web Docs — Documentación de referencia sobre CSS, fuentes, imágenes, filtros, fondos y propiedades de presentación.

Google Fonts — Documentación oficial para incorporación y utilización de fuentes web.

Font Awesome — Documentación oficial para incorporación y utilización de sus bibliotecas de iconos.